Milestone 3: Model or System Development and Alpha Application

Course: CIS-5980

Track: AI Engineering

Team: Will Liu, Guadalupe Cantera, and Bryan Yang

Repository: https://github.com/BryanDYang/ai-capstone

1. Core capability development and development path

Development paths:

For Milestone 3, we focused on prompt engineering, structured outputs, retrieval-augmented generation (RAG), and lightweight orchestration rather than fine-tuning. Our goal was to make the different parts of our project work together more reliably, especially when extracting commitments, saving information across meetings, and retrieving supporting evidence.

We chose this direction because many of the issues we encountered weren't necessarily problems with the models themselves. They involved how information moved between transcription, extraction, storage, and task updates. We also don't have enough independently labeled meeting data to justify fine-tuning yet, so we wanted to improve our existing pipeline first.

What changed since Milestone 2:

In Milestone 2, we mainly evaluated transcription and task extraction separately. We could identify commitments from individual meetings, but we didn't have a reliable way to save those commitments, recognize them in later meetings, or track whether anything changed.

For this milestone, we focused on building that missing connection:

- **Database integration:** Added Supabase PostgreSQL and pgvector to store meetings, transcripts, decisions, tasks, and supporting evidence. This gives our app a persistent record that can be used across multiple meetings.
- **Structured extraction:** Added Pydantic schemas and transcript quote verification to keep extracted information consistent and make sure commitments are supported by something that was actually said.

- **Task tracking:** Added task approval, editing, dismissal, and reconciliation. When a similar commitment appears in a later meeting, our app can suggest updating an existing task instead of automatically creating a duplicate.
- **Project chat:** Added RAG so users can ask questions about previous meetings and receive answers connected to specific transcript evidence.
- **iOS integration:** Connected authentication, project membership, optional voice enrollment, and meeting recording and uploads through our native iOS application.
- **Follow-through features:** Added Apple Reminders export, capture-only follow-up emails, limited auto-approval for clear self-commitments, and manual Zoom summary imports.

One of our biggest priorities was making sure commitments weren't lost between meetings. These changes helped us move beyond generating a summary after each recording and toward actually tracking what happens to those commitments over time.

Decision log:

Decision	What We Tried or Considered	What Worked, Didn't Work, and What We Decided
Retrieval	Dense-only, sparse-only, and hybrid search	Dense-only ranked target sources higher, but both dense and hybrid retrieved 11/12 sources within the top eight. We kept hybrid for now because each approach found some evidence the other missed.
Search and chunking	Approximate vs. exact cosine search and different transcript window sizes	We chose exact cosine search within each project and speaker-turn windows up to 150 words to keep relevant context together.
Citation verification	Direct database IDs vs. temporary citation handles	We chose backend-verified citation handles to reduce fabricated citations. However, strict quote matching sometimes rejected valid evidence because of formatting differences.
Task reconciliation	Automatic merging vs. human review	Similarity scores weren't enough to confirm whether two commitments referred to the same task. We decided to have users review proposed updates while preserving task IDs and histories.
Auto-approval	Manual approval for every task vs. limited automation	We added auto-approval only for clear self-commitments from confirmed speakers without unresolved issues. Other tasks still require review.

Extraction reliability	Older Codex CLI installation	HTTP 429 errors interrupted extraction, so we pinned a working version and made the CLI configurable.
Audio uploads	WAV files	A large upload failed, so we switched to lossless FLAC and verified that the decoded audio matched the original.

2. Training and configuration details

Training details, if applicable:

We didn't train or fine-tune any models for this milestone. Instead, we focused on improving our prompts, retrieval configuration, and how the different components interact.

Most of the problems we found involved retrieving the right evidence, interpreting updates to existing tasks, or validating model outputs. We felt it made more sense to address these issues first rather than fine-tuning without enough labeled examples or a clear understanding of which errors training would actually solve.

We also wanted to establish a more reliable baseline before considering fine-tuning. Once we have more independently reviewed meeting examples, we can revisit whether fine-tuning would help with recurring extraction errors, rather than using it to address issues that may actually come from transcription, retrieval, or task reconciliation.

Configuration details, if applicable:

Component	Configuration
Transcription	Whisper MLX medium; FFmpeg converts audio to 16 kHz mono WAV
Speaker identification	pyannote/speaker-diarization-3.1 and WeSpeaker; 256-dimensional embeddings
Voice matching	Cosine similarity ≥ 0.60 and a margin ≥ 0.10 over the next match; thresholds are preliminary
Meeting extraction	Codex CLI requesting gpt-5.6-sol, or Claude; meeting-extraction-v2 prompt and Pydantic schemas
Zoom summaries	zoom-summary-v1 prompt
Chat	project-chat-v2 prompt with structured answers and citation verification
Embeddings	OpenAI text-embedding-3-small; 1,536 dimensions

Retrieval	Hybrid semantic and PostgreSQL full-text search; top 8 results, RRF constant 60, search depth 40 per method
Task similarity	First 384 embedding dimensions; similarity threshold 0.80; matches suggest updates but don't apply them
Backend and storage	FastAPI, Supabase PostgreSQL, pgvector, and Cloudflare Tunnel

Our pipeline starts by preparing the audio and generating a transcript. If speaker identification is enabled, the system also attempts to match speakers using enrolled voice samples. The transcript is then passed through our extraction prompt, which returns structured information about decisions, commitments, and supporting quotes.

Before saving the results, we verify that the quoted evidence appears in the transcript. For task reconciliation, we use similarity to identify possible matches, but users still review proposed updates rather than having the system automatically merge commitments.

For chat, our system retrieves relevant information from the selected project and uses it to generate an answer. Citations are checked against stored evidence before the answer is returned. If the system can't verify enough supporting information, it's designed to respond with "I don't know."

Reproducibility notes:

Our retrieval evaluation used repository commit `af3a4b8`. Dependencies are locked in `uv.lock`, and database migrations are stored in `supabase/migrations/`.

Reproducing our setup requires OpenAI, Supabase, and Hugging Face credentials, access to Codex CLI or Claude, and a separate CCB transcriber folder available from our team.

Our retrieval questions are stored in `tests/fixtures/rag/`, and `scripts/evaluate_rag.py` records retrieval rankings, timings, configuration details, and model outputs. We also kept our Milestone 2 baseline results under `docs/archive/milestone_2/results/` and `transcription_results.md` so we can compare our progress across milestones.

3. Ablations and comparisons

Ablation plan:

We wanted to see whether combining semantic and keyword search would improve how our chat feature retrieves information from previous meetings.

We compared two approaches:

- **Dense-only retrieval:** Uses embeddings and semantic similarity to find relevant transcript information.
- **Hybrid retrieval:** Combines semantic similarity with PostgreSQL full-text search using reciprocal rank fusion.

We tested both approaches using 12 supported questions from four AMI meetings (TS3005a–d), which contained 219 indexed transcript chunks. Eight questions searched within one meeting, while four searched across the project.

We kept the same dataset, embeddings, retrieval limit, prompts, and answer model. The only difference was whether full-text search was included. This helped us isolate the effect of the retrieval method.

Our questions and expected supporting sources were prepared with AI assistance and haven't been independently reviewed, so we're treating these results as an initial comparison rather than a final benchmark.

Results table:

Hit@K measures whether the designated supporting source appears within the top K results. MRR measures how highly that source is ranked, so a higher value means the expected evidence tends to appear earlier.

Metric	Dense-Only Retrieval	Hybrid Retrieval
Hit@1	9/12 (75.0%)	4/12 (33.3%)
Hit@3	10/12 (83.3%)	7/12 (58.3%)
Hit@8	11/12 (91.7%)	11/12 (91.7%)
MRR@8	0.794	0.519
Median retrieval time	87.3 ms	88.3 ms

Interpretation:

We expected hybrid retrieval to perform better because it combines semantic and keyword search, but dense-only actually ranked our target sources higher more often. Both approaches still retrieved 11 of the 12 sources within the top eight results, meaning keyword search didn't clearly improve overall retrieval in this test.

We also noticed that each approach missed different sources. Hybrid found a pricing decision that dense-only missed, while dense-only found a rubber-housing decision that hybrid missed. Interestingly, dense-only was still able to answer the pricing question using a different transcript chunk, even though it didn't retrieve the designated source.

Another issue came up with the LCD question. Hybrid retrieved the correct evidence, but the model added "Decision:" to its quote, which caused our citation validator to reject the answer.

This showed us that retrieving the expected source doesn't always mean the final answer will be correct or accepted by our system. For our project, answer accuracy and evidence verification matter just as much as retrieval ranking.

We're keeping hybrid for now because the two approaches retrieved some different evidence, but we want to compare human-reviewed answer quality before deciding which one works better overall.

4. Extended evaluation and target tests

Target test suite:

For this milestone, we expanded our evaluation beyond individual transcription and extraction results. We wanted to test whether our app could manage commitments across meetings and whether the features worked together as expected.

Our backend now has 319 passing automated tests covering task management, citation verification, project access, retrieval, and email handling. These tests help us check backend behavior, although they don't directly measure model accuracy.

We also tested eight short personal recordings focused on task approval, tentative commitments, deadlines, repeated tasks, and completion. In a separate text-based four-meeting sequence, our system preserved the same task ID through three updates and kept its completed status after reopening the app.

Finally, we ran the retrieval comparison from Section 3 and began evaluating generated chat answers. Together, these tests gave us a better picture of where our system is working and where we still need more evaluation.

Robustness:

We tested several situations that could realistically happen during recurring meetings, including tentative statements, repeated commitments, spoken deadlines, and task completions.

A statement like "maybe I'll..." correctly stayed pending rather than being treated as a confirmed commitment. A repeated task also didn't create a duplicate, showing that our reconciliation process can recognize some existing commitments.

However, we found issues with dates and completions. In one recording, Whisper transcribed "October 23" as "October 3," and the extracted task ended up missing its deadline. In another, our system missed a spoken completion update, although the user could still mark the task as done manually.

We also tested processing failures. Switching to lossless FLAC resolved a large audio upload issue, and updating our Codex CLI setup resolved an extraction problem. Some chat evaluation calls are still timed out, so that remains an open issue.

Bias, where applicable:

We haven't completed a formal bias evaluation yet, and this is an area where we recognize our testing is still limited. So far, we've mainly used the AMI Meeting Corpus and recordings from one teammate. While these have helped us test the basic functionality of our app, they don't fully represent the different voices, accents, speaking styles, and group sizes we would expect in actual research meetings.

One concern is that our transcription and speaker identification models might perform differently depending on who is speaking. For example, accents, overlapping conversations, or differences in speaking volume could cause the system to misidentify a speaker or miss part of a commitment. Since our app tracks tasks across multiple meetings, these errors could also affect who gets assigned a task or whether their progress is accurately recorded.

We also recognize that our app is currently English-only, so we haven't evaluated how it handles multilingual conversations or speakers switching between languages. This could limit how useful the app is for research groups with different language backgrounds.

To address these concerns, we plan to expand our testing with more participants, different recording conditions, and larger group discussions. We also want to compare transcription accuracy, speaker identification, and task extraction across these different scenarios to see whether certain types of speakers or conversations are more likely to produce errors.

For now, our app leaves speakers anonymous when it isn't confident in a match and allows users to review and correct extracted tasks. These safeguards help reduce the impact of mistakes, but we still need broader testing before we can say how consistently the system performs across different users.

Safety and hallucination, where applicable:

We focused on preventing unsupported information from becoming confirmed tasks or appearing as verified evidence.

Our extraction process checks supporting quotes against the transcript and leaves unclear details empty rather than guessing. Most task changes require user approval, and our backend tests include project isolation and excluding dismissed tasks from active results.

For chat, citations must match stored evidence. This helps reduce fabricated references, but we also found that the validator can be too strict. In the LCD example, the correct evidence was retrieved, but the model added a "Decision:" prefix, causing the answer to be rejected.

We prepared two unsupported questions to test whether the system would appropriately refuse to answer, but both runs timed out. We therefore haven't measured refusal accuracy yet.

Latency and cost:

We measured model response time and estimated API costs for three workloads: project-wide chat, single-meeting chat, and meeting extraction.

Workload	Completed Runs	Median Model + Validation Time	Average Input / Output Tokens	Estimated API Cost per Call
Project-wide chat question	6	5.08 seconds	17,432 / 117	$0.072
Single-meeting chat question	3	4.96 seconds	18,021 / 87	$0.042
Meeting extraction (TS3005a baseline)	1	48.2 seconds	28,106 / 1,355	~$0.14

Retrieval itself was much faster than generation, taking approximately 88 ms. Loading source text took 0.7–0.9 seconds, and embedding all 14 evaluation questions took 1.8 seconds total.

Our cost estimates use the October 21 API rates recorded in our draft for gpt-5.6-sol: $4 per million input tokens, $0.40 per million cached input tokens, and $20 per million output tokens. Embeddings were estimated at $0.02 per million tokens.

These estimates represent what equivalent API usage would cost, not actual payments, since we're currently using a subscription. They also exclude hosting, indexing, failed calls, and local transcription.

Our live chat evaluation had nine completed calls, three timeouts at 180 seconds, one interrupted call, and 15 that weren't run. Three completed dense-retrieval answer calls had a median time of 109 seconds, but we don't have enough completed runs to determine why those calls took longer.

We also haven't measured the full recording-to-result time through the iOS app, which we plan to include in a later evaluation.

Updated results table and error analysis:

Our October 20 recording tests focused on how our app handled different commitment scenarios.

Test Case	What We Tested	Result
A	Manual task approval	Task was already auto-approved, so this

		wasn't a clean manual approval test
B-positive	Clear self-commitment	Passed; task was auto-approved and one email was captured
B-negative	Tentative commitment	Passed; task stayed pending
C / D	Manual approval and calendar deadlines	Passed; dates were entered manually
E	Email recipient opt-out	Passed; no email was captured
F1	Repeated commitment	No duplicate task, but the date was transcribed incorrectly and the deadline was missing
F2	Spoken task completion	No completion candidate; manual completion and Undo worked
Relaunch	Task persistence	Passed; information was preserved

Five of the eight recording cases behaved fully as expected. We also captured four follow-up emails during testing, but none were sent to real inboxes.

These tests helped us identify where the next improvements should focus:

What Failed	Why We Think It Happened	What We'll Change
Missing deadlines	Whisper misheard a date, and extraction didn't preserve the deadline.	Improve date handling and make it easier for users to check deadlines against the original evidence.
Missed completions	Our extraction process focuses more on new commitments than updates to existing tasks.	Add a separate step for detecting and proposing task updates and completions.
Rejected citations	Exact quote matching rejected valid evidence because of extra formatting.	Improve citation formatting and validation without weakening evidence checks.
Retrieval ranking	Adding full-text search sometimes pushed designated sources lower.	Compare final answer quality using human-reviewed questions before changing retrieval settings.
Chat timeouts	Some model calls exceeded our time limit, but we haven't confirmed the underlying cause.	Investigate the timeouts and rerun the interrupted evaluation.

5. Alpha application and inference service

End-to-end user flow:

A user starts by signing in and creating or joining a project. They can optionally enroll their voice using three short recordings to help the system recognize them in future meetings.

From there, they can record or upload a meeting. Once processing finishes, they can review the generated transcript, summary, decisions, and suggested commitments. Users can approve, edit, or dismiss tasks before adding them to their project.

Those tasks are saved across meetings. If a similar commitment comes up later, our app can suggest an update to the existing task rather than creating a duplicate. Users can also manage their commitments through To-do and Calendar views and export tasks to Apple Reminders.

Our chat feature lets users ask questions about previous meetings and view the evidence supporting each answer. We also support manual Zoom summary imports. Follow-up emails are currently generated for testing but aren't sent to real recipients.

Interface:

We built a native SwiftUI iOS application organized into four tabs:

- **Meetings:** Record or upload meetings, review transcripts and summaries, and view extracted decisions.
- **Tasks:** Review commitments, approve suggested tasks, and manage deadlines through To-do and Calendar views.
- **Chat:** Ask questions about previous meetings and view supporting citations.
- **Profile:** Manage account settings and optional voice enrollment.

We also have a `labsync` CLI for testing the backend without using the iOS interface.

Inference service:

Our FastAPI backend runs on an Apple Silicon Mac and connects to the iOS app through Cloudflare Tunnel over HTTPS. Supabase handles authentication, project data, and storage.

When a recording is submitted, the backend prepares the audio using FFmpeg and processes it with Whisper MLX. If enabled, pyannote and WeSpeaker help identify speakers. The transcript is then sent to Codex CLI or Claude for structured extraction.

Extracted information is checked against transcript evidence before being saved in PostgreSQL. For chat, we use OpenAI embeddings, pgvector, and PostgreSQL full-text search to retrieve relevant information before generating answers with verified citations.

We also added an optional shared-backend setup using Supabase storage and job leases. This is meant to make processing less dependent on one machine, although we still need to test recovery between two live machines.

Run instructions:

Our repository README contains the full setup instructions. The steps below summarize how to run our alpha application on an Apple Silicon Mac.

```
brew install uv cloudflared
git clone https://github.com/BryanDYang/ai-capstone.git && cd ai-capstone
uv sync --locked --extra dev --extra audio --extra server --extra diarize
cp .env.example .env          # add OPENAI_API_KEY, HF_TOKEN, Supabase values
codex login
uv run --locked --env-file .env --extra server python
scripts/check_supabase.py
#   expected: "Supabase connection OK"
scripts/setup_tunnel.sh api-yourname.guardianagent.dev labsync-yourname
# Terminal 1
uv run --locked --env-file .env --extra audio --extra server --extra diarize \
 labsync serve --whisper-backend mlx --whisper-model medium --diarize
# Terminal 2
cloudflared tunnel run labsync-yourname
# Terminal 3
scripts/check_tunnel.sh api-yourname.guardianagent.dev
#   expected: every line says PASS
open ios/MeetingApp/MeetingApp.xcodeproj
```

Build and run the app, sign up, create a project, and upload `tests/fixtures/ami/TS3005a-90s-135s.wav`. **Expected output:** after processing, the meeting shows a transcript with speaker labels, a summary, and candidate action items with quotes. If you just want to use the app, you can skip all of this: ask Will for access to the team backend and only use Xcode (README "Option A").

To rerun the retrieval ablation:

```
uv run --no-sync --env-file .env python scripts/evaluate_rag.py \
 --output artifacts/evaluation/m3-rag-YYYY-MM-DD-run-N --answers
```

Known limitations and failure modes:

Our app still misses some spoken completion updates, and transcription errors can affect extracted deadlines. For now, users can review and correct tasks manually while we improve extraction.

Speaker-matching thresholds still need more evaluation, and chat refusal testing is incomplete because of model timeouts. Follow-up emails also remain capture-only until we're more confident in the generated content.

Our backend currently depends on an available Apple Silicon Mac. We plan to measure full processing time through the iOS app, finish our end-to-end demo, and test shared-backend recovery.

6. Model Card and System Card drafts

Model Card draft:

Intended Use

Our project is designed to help research groups keep track of decisions and commitments across recurring meetings. It combines transcription, structured extraction, task tracking, and evidence-backed chat so users can review what was discussed and follow up on outstanding work.

Uses Outside the Intended Scope

Our app isn't meant to replace official meeting records, evaluate individual performance, or make decisions on behalf of users. Since transcription and extraction can make mistakes, suggested commitments shouldn't be treated as confirmed without appropriate review.

Evaluation Summary

Our results show progress toward connecting the different parts of our system, but our evaluation is still limited. We need more real meeting recordings and human-reviewed tests before making stronger claims about accuracy.

Evaluation	Result
Whisper tiny / base WER (M2)	28.8% / 24.5%
Extraction proxy accuracy (M2, 24 synthetic examples)	Rules: 0.71; Codex: 0.90; Qwen3: 0.93
Dense-only retrieval (M3)	Hit@8: 11/12; MRR: 0.794
Hybrid retrieval (M3)	Hit@8: 11/12; MRR: 0.519

Personal recording tests (M3)	5 of 8 cases behaved fully as expected

Known Limitations and Mitigations

Our main limitations include transcription errors affecting deadlines, missed completion updates, limited English-language testing, and incomplete chat accuracy and refusal evaluations.

We reduce these risks by verifying evidence, requiring user review for most task changes, leaving uncertain details empty, and keeping speakers anonymous when voice matching isn't confident enough.

Another limitation is the possibility of uneven performance across different speakers and meeting environments. Since we haven't tested our system with a wide range of accents, languages, and group sizes, we can't yet determine whether certain users are more likely to experience transcription or speaker identification errors. This is especially important for our project because an incorrect speaker label could result in a task being assigned to the wrong person. We plan to expand our testing and continue requiring user review when ownership or other task details are uncertain.

System Card draft:

System Components and Data Flow

Our SwiftUI app connects to a FastAPI backend through Cloudflare Tunnel, while Supabase handles authentication, storage, and project data.

Meeting audio is transcribed, optionally processed for speaker identification, and passed through structured extraction. Verified information is stored in PostgreSQL and used for task tracking, reconciliation, and chat. Our retrieval system uses pgvector and full-text search to find relevant meeting evidence.

Safety Measures and Guardrails

We require consent for recordings and make voice enrollment optional. Project-level access controls limit which information users can access, and users can review, edit, or undo task changes.

Users can also revoke voice enrollment and delete project data. Our chat system verifies citations against stored evidence, and follow-up emails remain capture-only during testing.

Operational Constraints and Mitigations

Our backend currently depends on an available Apple Silicon Mac and locally downloaded transcription models. We addressed large audio upload failures by switching to FLAC and fixed an extraction issue by updating our Codex CLI setup.

We still need to investigate model timeouts, evaluate speaker matching more broadly, and test shared-backend recovery between two machines.

7. Weekly check-ins and blockers

Progress made:

Since Milestone 2, we've connected our iOS app to authentication, project management, meeting processing, voice enrollment, task tracking, and project chat. We also added cross-meeting reconciliation, follow-up email capture, and limited auto-approval.

We expanded our testing with 319 passing backend tests, a retrieval ablation, and targeted recording scenarios. These helped us identify specific issues that weren't as visible when we were testing components separately.

Top blockers or risks:

Our biggest blockers are missed completion updates, transcription errors affecting deadlines, and model timeouts during evaluation. We also still need human-reviewed answer results and a complete recorded demo.

Another limitation is backend availability, since meeting processing currently depends on an Apple Silicon Mac being online.

Planned next steps:

For the next milestone, we want to finish our interrupted evaluations, review chat answers with human reviewers, and record a complete iOS demo.

We'll also focus on improving task completion detection, expanding speaker-matching tests, and making backend processing more reliable.

We also want to expand our bias evaluation by testing recordings from more speakers and different meeting environments, particularly to understand whether transcription and speaker identification errors affect certain users or meeting scenarios more than others.

Earlier progress is recorded in the weekly journal.

8. For group only (non-graded)

Team members and roles:

Will — Backend and iOS Integration

Will has focused on connecting our iOS app to the backend, including Supabase authentication, database integration, project management, voice enrollment, speaker matching, and RAG chat. His work has helped connect our individual components into an application that users can interact with.

Bryan — Audio Processing, Testing, and Performance Evaluation

Bryan has focused on the audio processing and extraction pipeline, follow-through features, and evaluation. He recorded test audio and documented checks for automatic approval, tentative commitments, email opt-out, manual dates, duplicate handling, and completion/Undo with persistence. Four follow-up emails were captured during testing, but none were sent to real inboxes. He has also taken responsibility for the dense-only versus hybrid retrieval comparison, latency and cost measurements, and technical evaluation documentation.

Guadalupe — Data Research, Evaluation, and Milestone Documentation

Guadalupe has focused on meeting dataset research, data organization, and evaluation planning. For Milestone 3, she has also helped organize, write, and revise our report, making sure our technical decisions, evaluation results, and limitations are clearly explained and aligned with the rubric. Her upcoming work includes independently reviewing the eight transcript/output cases to identify unsupported, missing, or unclear information before comparing judgments with the team's existing findings.

Workplan for the next milestone with an owner for each task:

Task	Owner
Independently review the eight transcript/output cases in `docs/milestone_3/human_review.md` and document unsupported, missing, or unclear information	Guadalupe
Continue organizing evaluation findings and supporting the writing and revision of our milestone documentation	Guadalupe
Complete and document a clean iOS app walkthrough covering upload, task review, calendar, captured email, chat, and relaunch	Will
Record the walkthrough results, including failures or blocked steps, in `docs/evidence/result.md`	Will
Finalize the dense-only versus hybrid retrieval comparison, latency/cost measurements, and technical evaluation write-up	Bryan
Investigate automatic deadline extraction and missed spoken completion updates	Bryan, with Will
Address integration issues and verify that the complete app workflow runs	Will, with Bryan

reliably	
Review the remaining evaluation results and prepare our final demonstration and documentation	All team members

Blockers, dependencies or risks:

Our main technical blockers are inaccurate automatic deadline extraction, missed spoken completion updates, and occasional model timeouts. Although our follow-through tests confirmed several working features, we still need to improve how our system handles changes to existing commitments.

We also need to complete our independent human review and document a clean end-to-end app walkthrough. These checks will help us better understand where our system is reliable and where it still needs improvement.

Another risk is backend availability, since processing currently depends on an Apple Silicon Mac being online. For the next milestone, we'll need to coordinate testing and integration so we have enough time to address any issues before our final demonstration.

9. Individual Reflection (non-graded)

Team Member → Will

1. Reflective Journal: What You Worked On and What You Learned

I worked on the authentication, speaker labeling via voice cosine similarity, transcription diarization, and profile page. For the authentication, users can use their own email or log in using google authentication. After registering, users will go through a voice enrollment process where they will read off the script so that the system can encode the users voice into a vector embedding and when each meeting audio is transcribed and diarized, the speaker labeling will match the attendee’s voices with the users who are invited to the meeting project and use a cosine similarity to match the user’s voices. The profile page will hold all of the user’s profile and privacy content.

I thought that understanding how to match user voices to label speakers in a meeting audio was pretty interesting when I did my research on it. I did not realize I needed to use a vectorized database, in which thankfully supabase already provided for us so we just needed to extend the number of tables for that.

2. Lessons Learned: What You Would Do Differently Next Time

I thought maybe structuring the scaffolding part of the documentation could use some improvement. For example, I think having documentation on how an agent should read each

documentation before it builds would result in better and more consistent output. Additionally, I think adding more continuous development with .yaml files could work better too as it might not require the developers to manually test each button press or action item and let the system do that itself while going through a test checklist.

3. Contribution Check

Each team member contributed meaningfully to the capstone project all in equal but different ways. Bryan worked on testing, evaluation, and workspace documentation, while Guadalupe worked on the database schema implementation, data research, and milestone documentation.I worked on the backend and UI implementation throughout various systems integrated into this project like the authentication, audio/task extraction/transcription/summarization, and calendar integration.

Team Member → Bryan

1. Reflective Journal: What You Worked On and What You Learned

During this milestone, I focused on making Guardian Agent's backend and native app work together reliably and evaluating the complete user experience. I set up a recoverable shared backend, improved meeting playback and speaker review, and implemented evidence-backed task updates across meetings, including deadline changes, completion, and undo. I also added task follow-up email support and optional automatic approval with evidence checks.

Alongside implementation, I tested native app workflows, ran an AMI retrieval comparison, and organized the evaluation evidence and milestone documentation. I learned that passing component tests does not guarantee a reliable user experience. Testing complete workflows exposed problems with dates, stale UI state, and task updates that were harder to catch in isolation. I also learned to distinguish implemented functionality from behavior we had actually verified.

2. Lessons Learned: What You Would Do Differently Next Time

Next time, I would establish a small, complete acceptance workflow earlier, covering recording or upload, transcript review, task approval, cross-meeting updates, and follow-up. This would help us identify integration problems before expanding the feature set.

I would also prepare evaluation questions, expected task states, and independent review assignments earlier. Some evaluation and human-review work remained incomplete while implementation continued. Setting clearer owners and checkpoints would give us more time to investigate failures and compare approaches. Our retrieval comparison also reinforced the value of measuring design choices: hybrid retrieval did not outperform dense retrieval on our small development set.

3. Contribution Check

Yes, each team member contributed meaningfully, although our responsibilities and volume of repository activity differed. Will contributed substantially to backend and iOS integration, authentication, project membership, voice enrollment and speaker matching, and project chat with retrieval and citations. Guadalupe contributed database/schema work and data research in earlier milestones and completed an independent human review during this milestone. My work centered on integration, backend reliability, task continuity and follow-through, end-to-end testing, evaluation, and submission documentation.

Team Member → Guadalupe

1. Reflective Journal: What You Worked On and What You Learned

During this milestone, I helped review and edit our Milestone 3 documentation to make sure we were addressing the rubric requirements and accurately reflecting our progress. I also completed the independent human review of our test cases, focusing on whether the generated outputs were supported by the original transcripts. One thing I learned from this process is how important it is for our app to distinguish between tentative statements and actual commitments, since even small differences in wording can affect how tasks are identified.

2. Lessons Learned: What You Would Do Differently Next Time

Looking back, I think it would have been helpful to start the human review process earlier so we had more time to compare results and discuss any disagreements. I also think we could benefit from testing more realistic meeting scenarios, especially ones with multiple speakers and less structured conversations. This would help us better understand where our app struggles before moving further into development.

3. Contribution Check

Yes, everyone contributed meaningfully to this milestone. Bryan and Will focused primarily on the technical development, testing, and improvements to our app, while I contributed to the documentation, evaluation, and independent human review. We each worked on different parts of the project and communicated throughout the milestone to make sure everything came together!

10. Reference

- **AMI Meeting Corpus:** https://groups.inf.ed.ac.uk/ami/corpus/
- **OpenAI API and Embeddings:** https://platform.openai.com/docs/
- **Whisper:** https://github.com/openai/whisper
- **MLX Whisper:** https://github.com/ml-explore/mlx-examples/tree/main/whisper
- **pyannote.audio:** https://github.com/pyannote/pyannote-audio
- **WeSpeaker:** https://github.com/wenet-e2e/wespeaker
- **Supabase:** https://supabase.com/docs
- **pgvector:** https://github.com/pgvector/pgvector

- **FastAPI:** https://fastapi.tiangolo.com/
- **Cloudflare Tunnel:** https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/
- **SwiftUI:** https://developer.apple.com/documentation/swiftui/

11. GitHub Repository Link

https://github.com/BryanDYang/ai-capstone